ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

———— 綾鳥テキスタイル株式会社 14 ヶ月の戦争に投入された Python 実装

本書は、ケーススタディ本編『生存の幾何学』に登場する Python コード断片を、
パイプラインの 10 ステップ構造に沿って整理し、
各コード片の数学的背景と運用上の意図を解説した実装ガイドである。

Étale Cohomology

2026 年 4 月

本書の位置づけ

本書はシリーズ全体の補助文書 03 である。関連文書として以下がある。

- 本編：『生存の幾何学 —— 綾鳥テキスタイル株式会社 14 ヶ月の戦争』
- 補助文書 01：本編と GI 理論の対応関係マッピング
- 補助文書 02：数学的基盤
- **本書（補助文書 03）：実装スクリプト集**
- 補助文書 04：業界知識

掲載コードについて

本書に掲載されたコード断片は、本編に登場した擬似コードを忠実に再録したものである。本編の擬似コードは、教育目的での読みやすさを優先しているため、そのままでは実行できない箇所（外部関数の呼び出し、省略された数学的処理など）を含む。これらは数式とコードの橋渡しを意図した記述であり、実運用では各組織の環境に合わせた完全実装が必要となる。

ライセンスとクレジット

© 2026 Étale Cohomology. All rights reserved.

本書および関連シリーズはオープンソースとして公開されている。

Contents

1 はじめに **3**
1.1 パイプライン全体像 3
1.2 本書の構成 3
1.3 各コード片の構成 3
1.4 推奨ライブラリ 3

2 ステップ 1：データ収集と前処理 **4**
2.1 データロードと指標定義 4
2.2 レジームシフトを考慮した前処理関数 5

3 ステップ 2：VAE による多様体構築 **7**
3.1 固有次元数の推定 7
3.2 命題 1.1 の検証 —— 第 1 回試行（不合格） 8
3.3 制約付き Neural ODE による合成境界データ 9
3.4 命題 1.1 の検証 —— 第 2 回試行（合格） 10

4 ステップ 3：プルバック計量の計算 **11**
4.1 プルバック計量関数 11

5 ステップ 4：スカラー曲率の計算 **13**
5.1 スカラー曲率関数 13

6 ステップ 5：リー微分とリー括弧 **15**
6.1 リー微分（単一政策の構造変革効果） 15
6.2 リー括弧（複数政策の創発効果） 16

7 ステップ 6：MC ドロップアウトによる不確実性推定 **18**
7.1 MC ドロップアウト関数 18

8 ステップ 7：SHAP と DML **20**
8.1 SHAP と DML の併用 20

9 ステップ 8：多様体上の最適制御 **22**
9.1 ManifoldOptimalControl クラス 22
9.2 制御オプションの定義と最適経路求解 23

10 幾何学的拡張 **26**
10.1 層理論による偽造品検知 26
10.2 ベクトル場（DPP 規制 / 敵対的買収） 27
10.3 手術理論：サプライチェーン再構築 28
10.4 TDA：パーシステント・ホモロジー 30
10.5 ゲージ変換：多様体の再ランダム化 31
10.6 応力テンソル分析：レジリエンス測定 32

11 長期シミュレーション：シンプレクティック幾何学 **34**
11.1 Störmer-Verlet シンプレクティック積分器 34

12 14 体 AI エージェント配備 **37**
12.1 エージェント・ロスター 37

13 形式検証と ZKML **41**
13.1 Coq 証明と Z3 SMT 検証 41
13.2 ZKML 証明生成 42

A 推奨ライブラリ一覧 **44**
A.1 基本数値計算・データ処理 44
A.2 深層学習と Neural ODE 44
A.3 説明可能性と因果推論 44

A.4 位相的データ解析 44
A.5 形式検証 44
A.6 ZKML 44
A.7 可視化（本書のコードでは直接使用しないが、本編で言及） 44
A.8 インストールコマンド 44
B コード索引 **46**
B.1 パイプラインステップ別の主要コード片 48
B.2 結語 48

1　はじめに

本書は、ケーススタディ本編『生存の幾何学』に登場する Python コード断片を、 ジオメトリック・インテリジェンス（GI）理論の 10 ステップ・パイプラインに沿って整理し、 各コードの数学的意味と運用上の役割を解説する実装ガイドである。

1.1　パイプライン全体像

GI 理論の 10 ステップ・パイプラインは以下の通り。

1. **データ収集と前処理** —— 経営指標を収集、レジームシフトを保存しつつ欠損値を補完
2. **VAE による多様体構築** —— 高次元データを潜在空間に圧縮、命題 1.1 の 3 条件を検証
3. **プルバック計量の計算** —— 多様体上の各点で「距離の測り方」を定義
4. **曲率の計算** —— スカラー曲率による安定性／不安定性の可視化
5. **リー微分とリー括弧** —— 政策が環境構造に与える影響、複数政策の相互作用を定量化
6. **MC ドロップアウト** —— すべての幾何学的量に信頼区間を付与
7. **SHAP + DML** —— 要因分解（SHAP）と因果効果推定（DML）
8. **Neural ODE + 最適制御** —— ポントリャーギンの最大値原理による軌道予測と最適政策
9. **品質チェック** —— 全ステップの品質指標の統合チェック
10. **レポート生成** —— 経営陣向け意思決定支援レポート

1.2　本書の構成

- 第 2 章 —— 第 7 章：パイプラインのステップ 1 —— ステップ 6 の実装
- 第 8 章：ステップ 7（SHAP と DML）
- 第 9 章：ステップ 8（多様体上の最適制御）
- 第 10 章：幾何学的拡張（層理論、TDA、ゲージ変換、応力テンソル）
- 第 11 章：形式検証と ZKML
- 付録 A：推奨ライブラリ一覧
- 付録 B：コード索引

1.3　各コード片の構成

各節は以下の三段構成で記述される。

1. **背景** —— そのコードが本編のどの危機局面で登場し、どんな問いに答えるものか
2. **コード** —— 本編の Python 実装を構文ハイライト付きで掲載
3. **解説** —— 数学的意味、実装上の注意点、本編における結果

1.4　推奨ライブラリ

本書のコード片は以下のライブラリを前提とする。詳細なバージョン情報は付録 A を参照。

- `numpy`, `scipy`, `pandas`, `scikit-learn` —— 数値計算・データ処理
- `torch`, `torchdiffeq` —— ニューラルネットワークと Neural ODE
- `shap` —— SHAP 説明可能性
- `ripser`, `persim` —— パーシステント・ホモロジー（TDA）
- `z3-solver` —— SMT ソルバー（形式検証）
- `ezkl` —— ZKML 証明生成

2　ステップ 1：データ収集と前処理

パイプラインの起点。本編における **第 1 月**（2026 年 9 月）の作業に対応する。

2.1　データロードと指標定義

背景

綾鳥テキスタイルの経営環境を構成する 100 の指標を 8 カテゴリに分類し、5 地域（JP・EU・US・SEA・ME）× 月次 8 年分のデータを統合する。本編冒頭で CFO 長谷川が「100 指標、8 年分、データ品質はまちまち」と指摘した懸念に応える前処理パイプラインの第 1 段階。

コード

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer

# ===== ステップ 1：綾鳥データの収集と前処理 =====

# カテゴリ 1：経営実績データ
# 地域：JP（日本）、EU、US、SEA（東南アジア）、ME（中東）
# 期間：月次、2018 年 4 月〜2026 年 8 月 = 101 ヶ月 × 5 地域 = 505 データポイント／指標

business_indicators = [
    # 売上指標（7）
    'revenue_total',          # 総売上高（十億円）
    'revenue_technical_textile', # 機能性テキスタイル売上
    'revenue_fashion_brand',   # ファッションブランド売上
    'revenue_industrial',      # 産業用テキスタイル売上
    'gross_margin',           # 売上総利益率（%）
    'operating_margin',        # 営業利益率（%）
    'market_share_local',      # 国内市場シェア（%）

    # コスト指標（8）
    'cogs_raw_cotton',         # 原綿コスト（十億円）
    'cogs_dye_chemicals',      # 染料コスト
    'cogs_polyester',          # ポリエステルコスト
    'logistics_cost',          # 物流コスト
    'marketing_spend',         # マーケティング費
    'rd_spend',               # 研究開発費
    'labor_cost',             # 人件費
    'energy_cost',            # エネルギーコスト（染色にとって重要）

    # 生産指標（7）
    'factory_utilization_vietnam',   # ベトナム工場稼働率（%）
    'factory_utilization_turkey',    # トルコ工場稼働率（%）
    'factory_utilization_italy',     # イタリア（コモ）工場稼働率（%）
    'factory_utilization_japan',     # 日本工場稼働率（%）
    'inventory_turnover',            # 棚卸資産回転率
    'defect_rate',                  # 不良率（ppm）
    'supply_chain_lead_time',       # 平均リードタイム（日）
]

regions = ['JP', 'EU', 'US', 'SEA', 'ME']
```

```
# 綾鳥の SAP システムからデータをロード・統合
all_business_data = {}
for region in regions:
    df = pd.read_csv(f'ayatori_data/business_{region}_monthly.csv')
    all_business_data[region] = df

df_business = pd.concat(
    [all_business_data[r].assign(region=r) for r in regions]
)

print(f"経営データ：{df_business.shape[0]}行 × {df_business.shape[1]}列")
print(f"期間：{df_business['date'].min()} — {df_business['date'].max()}")
print(f"欠損値：{df_business.isnull().sum().sum()}セル")
```

解説

原則 4：材料の品質が全て（ショートブレッドの教訓）。 ステップ 2 以降の VAE 多様体構築、曲率計算、最適制御の品質は、 すべてこのステップで読み込まれるデータの品質に上限を規定される。 バターが酸化していれば、どれだけ精緻なオーブンでもショートブレッドは焼けない。

指標カテゴリの設計判断として、本ケーススタディでは 売上（7）・コスト（8）・生産（7）の 22 指標を「経営実績」として定義した。 これは綾鳥のような製造業企業に固有の構成であり、 別の業界（金融、エネルギー、安全保障）に適用する際は指標体系自体を再設計する必要がある。 GI 理論の **業界非依存性**（命題 1.1 が $J^\top J$ の代数的構造のみから従うこと）が 意味するのは、計算の数学的健全性が業界によらず保証されるということであり、 どの指標を選ぶかの設計判断が業界によらないということではない。

2.2　レジームシフトを考慮した前処理関数

背景

本編で最初の重大な議論を引き起こしたデータ処理上の判断。 COVID-19 期間を「異常データ」として補間で平滑化するか、それとも「構造的断絶」として保存するか? リン（数理アナリスト）の指摘 —— 「補間してしまえば VAE が突然のレジーム変化の可能性を過小評価する」 —— を受けて確立された処理プロトコル。

コード

```
# 綾鳥データの前処理プロトコル
def preprocess_ayatori_data(df):
    """
    レジームシフトを考慮した前処理
    - 非危機期の欠損値には KNN 補完
    - COVID-19、サプライチェーン危機のレジームシフト・マーカー
    - ロバストスケーリングによる標準化
    """

    # 1. レジームシフト期間を特定する（これらは補完しない）
    regime_shifts = {
        'covid_wave1': ('2020-01', '2020-06'),
        'suez_canal': ('2021-03', '2021-04'),
        'ukraine_war': ('2022-02', '2022-06'),
```

```
    }

    # 2. レジームシフト行にマークを付ける
    df['is_regime_shift'] = False
    for name, (start, end) in regime_shifts.items():
        mask = (df['date'] >= start) & (df['date'] <= end)
        df.loc[mask, 'is_regime_shift'] = True

    # 3. レジームシフト以外の欠損値にのみ KNN 補完を適用
    normal_mask = ~df['is_regime_shift']
    imputer = KNNImputer(n_neighbors=5, weights='distance')

    numeric_cols = df.select_dtypes(include=[np.number]).columns
    df.loc[normal_mask, numeric_cols] = imputer.fit_transform(
        df.loc[normal_mask, numeric_cols]
    )

    # 4. レジームシフト期間：利用可能な実測値を使い、
    #    残りの欠損には regime_shift_flag を付ける（補完しない）
    remaining_missing = df.isnull().sum().sum()
    print(f"KNN 補完後：{remaining_missing}セルがまだ欠損")
    print(f"  （これらはレジームシフト期間の欠損 —— 意図的に保存）")

    # 5. ロバスト標準化（中央値／IQR ベース、外れ値に頑健）
    from sklearn.preprocessing import RobustScaler
    scaler = RobustScaler()
    df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

    return df, scaler
```

解説

レジームシフトを保存する設計 は GI 理論パイプラインの根本的特徴である。 COVID（2020 Q1 —— Q2）、スエズ運河封鎖（2021 Q1）、ウクライナ戦争（2022 Q1 —— Q2）の 3 期間を `is_regime_shift` フラグでマークし、これらの期間内の欠損値は **KNN 補完を適用しない**。

これは技術的には「補間によって不連続点が滑らかにされる」ことを防ぐためだが、 理論的にはより深い意味を持つ：VAE が学習する潜在多様体は 「レジーム変化が起こりうることを構造的に知っている」多様体でなければならない。 平時データのみで学習された多様体は、危機が来た瞬間に 多様体の外部の点（モデル外挿領域）にエージェントを押し出してしまう。

RobustScaler の選択：標準スケーラ（z-score 正規化）ではなく、 中央値と IQR（四分位範囲）を使うロバストスケーラを使用している。 極端な外れ値（フラグ付きで保持された 147 件）が存在するデータでは、 平均と標準偏差ベースの正規化は外れ値に引っ張られて全体を歪めるためである。

3　ステップ 2：VAE による多様体構築

本編における **第 1 月後半**（2026 年 9 月 15 日 —— 30 日）の作業。 本ケーススタディの最初の数学的危機が発生した局面。

3.1　固有次元数の推定

背景

VAE の潜在次元 d を決定するための前段階。 100 の生指標の背後にある「実質的な自由度」が何次元かを Levina-Bickel（2004）の 最尤推定法で求める。

コード

```python
import torch
import torch.nn as nn
from torch.distributions import Normal, kl_divergence

# ===== ステップ 2：綾鳥経営環境多様体のための VAE =====

# まず：固有次元数を推定する
from sklearn.neighbors import NearestNeighbors
from scipy.stats import linregress

def estimate_intrinsic_dimension(X, k_range=(5, 50)):
    """
    固有次元数の最尤推定。
    手法：Levina-Bickel (2004)
    """
    n_samples = X.shape[0]
    dims = []

    for k in range(k_range[0], k_range[1]):
        nn = NearestNeighbors(n_neighbors=k+1).fit(X)
        distances, _ = nn.kneighbors(X)

        # 各点での MLE 推定
        T_k = distances[:, k] # k 番目の最近傍距離
        local_dims = []
        for i in range(n_samples):
            if T_k[i] > 0:
                log_ratios = np.log(T_k[i] / distances[i, 1:k])
                d_hat = (k - 1) / np.sum(log_ratios)
                local_dims.append(d_hat)
        dims.append(np.median(local_dims))

    return np.median(dims)

# 綾鳥の経営環境の固有次元数を推定
d_hat = estimate_intrinsic_dimension(X_normalized)
print(f"推定固有次元数：{d_hat:.1f}")
```

解説

ケーススタディでの推定結果は $\hat{d} = 8.3$。 これを受けてカイトは VAE の潜在次元を 8 に設定し、「100 の生指標があるにもかかわらず、綾鳥の経営環境における実質的な自由度は約 8」というリンの解釈に至る。

固有次元数の推定値は VAE のアーキテクチャ選択を導くが、**推定値そのものを盲信してはいけない**。 ケーススタディ B（国家安全保障会議）では $\hat{d} = 8$ を採用しているが、 これは推定値だけでなく 命題 1.1 の 3 条件が $d \in \{7, 8, 9\}$ の各候補で最良スコアを示すかを比較した結果でもある。 本コードは「決定的な値を 1 つ返す」のではなく「アーキテクチャ探索の起点を提供する」役割を持つ。

3.2 命題 1.1 の検証 —— 第 1 回試行（不合格）

背景

本編における最初の数学的危機。VAE を 14 時間かけて訓練した結果、命題 1.1 の 3 条件のうち 2 条件が不合格となる。 ロンドンのウォールームが静まり返り、カイトの顔色が青くなる場面。

コード

```python
# 命題 1.1 検証結果 —— 第 1 回試行

# 条件 1（再構成忠実度）：
mse_reconstruction = 0.0073
threshold_epsilon1 = 0.01
condition_1 = mse_reconstruction < threshold_epsilon1
print(f"条件 1：MSE = {mse_reconstruction:.4f} < {threshold_epsilon1} → "
      f"{'合格 ✓' if condition_1 else '不合格 ☒'}")

# 条件 2（潜在空間の滑らかさ）：
# テスト：k 最近傍保存率
knn_preservation_rate = 0.84
threshold_epsilon2 = 0.90
condition_2 = knn_preservation_rate >= threshold_epsilon2
print(f"条件 2：KNN 保存率 = {knn_preservation_rate:.2f} >= "
      f"{threshold_epsilon2} → "
      f"{'合格 ✓' if condition_2 else '不合格 ☒'}")

# 条件 3（ヤコビアンの非退化性）：
min_singular_value = 0.0002
threshold_epsilon3 = 0.001
condition_3 = min_singular_value >= threshold_epsilon3
print(f"条件 3：最小特異値 = {min_singular_value:.4f} >= "
      f"{threshold_epsilon3} → "
      f"{'合格 ✓' if condition_3 else '不合格 ☒'}")
```

解説

命題 1.1 の 3 条件は、構築された潜在空間が 単なる点群ではなく 微分計算が可能な滑らかな多様体として機能する保証を与える。

- **条件 1**（再構成忠実度）：MSE $< 0.01 \rightarrow$ **合格** ✓（0.0073）
- **条件 2**（潜在空間の滑らかさ）：KNN 保存率 $\geq 0.90 \rightarrow$ **不合格** ☒（0.84）

・**条件 3**（ヤコビアンの非退化性）：最小特異値 ≥ 0.001 → **不合格** ☒（0.0002）

リンが t-SNE 射影を可視化したところ、データは **3 つの明確なクラスター**（COVID 前・COVID 撹乱・COVID 後ニューノーマル）に分離していた。 倉田教授の言葉：「データには 3 つの連結成分があり、VAE はそれを無理に 1 つの連結多様体に押し込もうとしている」。

これは **位相的な問題**であり、データそのものの問題である。アーキテクチャをいくら調整しても解決しない。 解決には 遷移期間のデータを補強する必要があった。

3.3　制約付き Neural ODE による合成境界データ

背景

第 1 回不合格を受けての対応策。レジーム間の遷移期間（COVID 前 → 中 → 後）の 日次粒度データを綾鳥から追加調達するのと並行して、Neural ODE による **物理的制約を尊重した補間**で境界データを生成する。

リンの懸念：「合成データは『多様体が幻想になる』ことそのものではないか」。 リームの応答：「捏造データはランダムノイズの整形だが、これは制約付き補間 —— 既知の始点・終点間の最も蓋然性の高い経路を埋めるもの」。

コード

```
# 物理情報に基づく遷移補間
import torchdiffeq

class ConstrainedBusinessDynamics(nn.Module):
    """経営の物理法則を尊重する Neural ODE：
    - 売上の連続性（ワープしない）
    - 生産能力制約（稼働率 ∈ [0, 100%]）
    - 総資産の保存
    """
    def __init__(self, latent_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh(),
            nn.Linear(128, latent_dim)
        )

    def forward(self, t, z):
        dz = self.net(z)
        # 射影による生産能力制約の適用
        # 工場稼働率の次元は[0, 1]に留まる
        # 売上の次元は非負に留まる
        return dz

# 合成遷移軌道の生成
dynamics = ConstrainedBusinessDynamics(latent_dim=8)
# 既知の始点・終点状態と境界条件で訓練
# ...
# 遷移あたり 100 本の合成日次軌道を生成
synthetic_trajectories = torchdiffeq.odeint(
    dynamics, z_start, t_span, method='dopri5'
)
```

解説

ホイットニーの拡張定理（倉田教授による数学的正当化）：十分に滑らかな境界条件が与えられれば、それらを補間する滑らかな関数が存在する。本コードは Neural ODE をそのような滑らかな補間関数として用いる。

実装上の重要な制約：

- 売上の連続性（瞬間移動しない）
- 工場稼働率は $[0, 1]$ に留まる（射影によって境界外を抑制）
- 総資産の保存（局所的な保存則）

'tanh' 活性化（ReLU ではなく）を採用しているのは GI 理論の重要な設計判断。ReLU は C^0 連続だが C^1 ではない。リーマン曲率テンソルの計算には計量の 2 階偏微分が必要であり、それは VAE デコーダ f の 3 階微分性（C^3 級）を要求する。ReLU では原理的に曲率計算が破綻する。

3.4 命題 1.1 の検証 —— 第 2 回試行（合格）

背景

5 日間の集中データ手術（日次データ取得 —— SNS センチメント拡充 —— 合成境界データ生成 —— VAE 再訓練）を経て、命題 1.1 を再実行する。

コード

```
# ===== 命題 1.1 再検証 —— 第 2 回試行 =====

# 条件 1（再構成忠実度）：
mse_reconstruction = 0.0051
print(f"条件 1：MSE = {mse_reconstruction:.4f} < 0.01 → 合格 ✓")

# 条件 2（潜在空間の滑らかさ）：
knn_preservation_rate = 0.937
print(f"条件 2：KNN 保存率 = {knn_preservation_rate:.3f} >= 0.90 → 合格 ✓")

# 条件 3（ヤコビアンの非退化性）：
min_singular_value = 0.0089
print(f"条件 3：最小特異値 = {min_singular_value:.4f} >= 0.001 → 合格 ✓")
```

解説

3 条件すべて合格：

- 条件 1：MSE = 0.0051（前回 0.0073 から改善）
- 条件 2：KNN 保存率 = 0.937（前回 0.84 から大幅改善）
- 条件 3：最小特異値 = 0.0089（前回 0.0002 から改善）

リンの解釈：「日次の遷移データが 3 つのレジームクラスターの間の橋を埋めた。擦り切れた橋でつながった 3 つの島ではなく、多様体はレジーム間の滑らかな遷移を持つ 1 つの連続した曲面になった。VAE は、COVID が『異なる世界への瞬間移動』ではなかったことを学習した —— それは既存の経営環境の急速だが連続的な変形だった」。

これでようやくステップ 3 以降（プルバック計量、曲率計算、最適制御）に進める。**多様体が数学的に健全であることが、すべての下流計算の前提条件である。**

4　ステップ 3：プルバック計量の計算

本編における **第 2 月**（2026 年 10 月 1 日 —— 15 日）の作業。「風景が姿を現す」章のテーマ。

4.1　プルバック計量関数

背景

VAE デコーダ f のヤコビアン $J(z)$ を介してデータ空間のユークリッド計量を潜在空間に引き戻し、潜在空間上の各点で「距離をどう測るか」を定義する。これが GI 理論の幾何学的計算すべての基礎となる。

コード

```
# ===== ステップ 3：プルバック計量の計算 =====

def compute_pullback_metric(decoder, z, epsilon=1e-4):
    """
    多様体上の点 z におけるプルバック計量テンソル g_ij を計算する。

    プルバック計量は多様体上で「距離をどう測るか」を定義する。
    「売上」方向に 1 単位動くことが、「センチメント」方向に 1 単位動くよりも
    はるかに大きな経営状態の変化に対応するかもしれない、という事実を捉える。

    g_ij(z) = J(z)^T @ J(z)
    ここで J(z) は点 z におけるデコーダのヤコビアン。
    """
    z_tensor = torch.tensor(z, dtype=torch.float32, requires_grad=True)

    # 有限差分によるヤコビアン計算（VAE デコーダには autograd より安定）
    J = np.zeros((decoder.output_dim, len(z)))
    for j in range(len(z)):
        z_plus = z.copy()
        z_minus = z.copy()

        z_plus[j] += epsilon
        z_minus[j] -= epsilon
        J[:, j] = (
            decoder(torch.tensor(z_plus)).detach().numpy()
            - decoder(torch.tensor(z_minus)).detach().numpy()
        ) / (2 * epsilon)

    # プルバック計量：g = J^T @ J
    g = J.T @ J
    return g


# 多様体上の 1,000 サンプル点で計量を計算
sample_points = vae.encode(X_test)  # テストデータを潜在空間にエンコード
metrics = [compute_pullback_metric(vae.decoder, z) for z in sample_points]

# 計量テンソルの固有値分析
for i, g in enumerate(metrics):
    eigenvalues = np.linalg.eigvalsh(g)
    condition_number = eigenvalues[-1] / eigenvalues[0]
    if condition_number > 1000:
```

```
        print(f"警告：点{i}の条件数は{condition_number:.0f}")
        print(f" → 計量がほぼ退化。この領域は信頼性が低い可能性がある。")
```

解説

数式の意味：プルバック計量

$$g_{ij}(z) = \sum_{a=1}^{n} \frac{\partial f^a}{\partial z^i}(z) \frac{\partial f^a}{\partial z^j}(z) = (J^\top J)_{ij}$$

は、潜在空間上の微小な移動が、データ空間でどれだけの変化を引き起こすかを各点・各方向で定量化する。

固有値分析の意味： $g_{ij}(z)$ の最大固有値方向は「敏感な方向」（その方向に動くとデータ空間で大きな変化）、 最小固有値方向は「鈍感な方向」。両者の比（**条件数**）は、その点における多様体の 異方性を表す。

ケーススタディの結果：

- 平均条件数：47.3
- 最大条件数：312.8（点 847、2022-03-15、ウクライナ危機開始日）

リンの解釈：「あの時点の経営環境は本質的に 1 次元だった —— すべてがエネルギー価格と制裁に関わることだった」。 1 つの方向が他のすべてを 300 倍の係数で支配しているとき、その時点の経営は実質的に 1 自由度系である。

有限差分 vs 自動微分：本コードは有限差分を採用している。 理論的には PyTorch の `autograd` で計算する方が精度が高いが、 VAE デコーダのような複雑な計算グラフでは、特異点近傍で数値不安定性が発生することがある。 有限差分は ϵ を適切に選べば（典型的には 10^{-4}）、安定で再現可能な結果を与える。

5　ステップ 4：スカラー曲率の計算

5.1　スカラー曲率関数

背景

プルバック計量から、クリストッフェル記号 Γ^k_{ij} を経由してリーマン曲率テンソル、 リッチテンソル、最終的にスカラー曲率 R を計算する。 スカラー曲率は 多様体のどこが安定で、どこが危険なほど急峻か を要約する単一の量である。

コード

```
# ===== ステップ 4：スカラー曲率の計算 =====

def compute_scalar_curvature(decoder, z, epsilon=1e-4):
    """
    点 z におけるスカラー曲率 R を計算する。

    スカラー曲率は空間がどれだけ曲がっているかの「要約統計量」。
    - R > 0：局所的に「球状」（収束的、安定）
    - R ≈ 0：局所的に「平坦」（中立）
    - R < 0：局所的に「鞍型」（発散的、不安定）

    大きな負の曲率 = 危険地帯（小さな摂動が増幅される）
    """
    dim = len(z)

    # 点 z および近傍点での計量テンソルを計算
    g = compute_pullback_metric(decoder, z, epsilon)

    # クリストッフェル記号 Γ^k_ij を計算
    christoffel = np.zeros((dim, dim, dim))
    g_inv = np.linalg.inv(g)

    for i in range(dim):
        z_plus_i = z.copy(); z_plus_i[i] += epsilon
        z_minus_i = z.copy(); z_minus_i[i] -= epsilon
        dg_di = (compute_pullback_metric(decoder, z_plus_i) -
                 compute_pullback_metric(decoder, z_minus_i)) / (2 * epsilon)

        for j in range(dim):
            for k in range(dim):
                # Γ^k_ij = (1/2) g^{kl} (∂_i g_{jl} + ∂_j g_{il} − ∂_l g_{ij})
                christoffel[k, i, j] = 0.5 * sum(
                    g_inv[k, l] * (
                        dg_di[l, j] + dg_di[j, l]
                        # — ∂_l g_{ij} 項は（計量の偏微分の対称化処理）
                    )
                    for l in range(dim)
                )

    # リーマン曲率テンソル → リッチテンソル → スカラー曲率
    #（自動微分を用いた簡略化計算）
    R_scalar = compute_ricci_scalar_from_christoffel(
        christoffel, g_inv, z, epsilon
    )
```

```
    return R_scalar

# 多様体全体のスカラー曲率を計算
curvature_map = {}
for i, z in enumerate(sample_points):
    R = compute_scalar_curvature(vae.decoder, z)
    date = dates[i]
    curvature_map[date] = R

# 最も危険な領域（大きな負の曲率）を探す
dangerous_points = [(d, R) for d, R in curvature_map.items() if R < -5.0]
dangerous_points.sort(key=lambda x: x[1])

print("多様体上の不安定領域 TOP 10：")
for date, R in dangerous_points[:10]:
    print(f" {date}：スカラー曲率 R = {R:.2f}")
```

解説

スカラー曲率の物理的意味：

- $R > 0$ —— 局所的に「球状」（測地球の体積がユークリッドより小さい、収束的、**安定**）
- $R \approx 0$ —— 局所的に「平坦」（中立）
- $R < 0$ —— 局所的に「鞍型」（測地球の体積がユークリッドより大きい、発散的、**不安定**）

ケーススタディの結果（多様体上の不安定領域 TOP 10）：

- 2020-03-15：$R = -47.31$（COVID グローバル・ロックダウン）
- 2022-02-28：$R = -38.92$（ウクライナ戦争勃発）
- 2026-08-26：$R = -22.17$（**現在**：エンティティ・リスト＋サイバー探索）
- 2021-03-25：$R = -18.44$（スエズ運河封鎖）

2026 年 8 月の現在地点は、8 年分のデータの中で 3 番目に深い曲率の谷にある。 ソフィアのヴィジュアライゼーションは「現在位置を不吉な深いインディゴ色」として描き、 ユキの叫び：「もう崖っぷちにいるんですか? まだ本格的な攻撃すら受けていないのに?」を引き出す。

擬似コード断片の注記：本コードのクリストッフェル記号計算部分には $\partial_l g_{ij}$ 項の対称化処理が一部省略されている（本編の擬似コードに忠実に再録）。 完全な実装では、z の各成分に対する数値微分をループで実行し、 $g_{kl}(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij})$ の全項を計算する必要がある。

曲率テンソルからスカラー曲率への縮約： $\mathrm{Ric}_{ij} = R^l_{ilj}$（リッチテンソル、曲率テンソルの縮約）、 $R = g^{ij}\mathrm{Ric}_{ij}$（スカラー曲率、計量で再縮約）。 これらは関数 `compute_ricci_scalar_from_christoffel`（コード中で参照）に集約されている。

6　ステップ 5：リー微分とリー括弧

本編における **第 2 月後半**（2026 年 10 月）の作業。 **ベトナム工場へのランサムウェア攻撃**（10 月 3 日午前 2 時 47 分）の直後、 48 時間以内に行われた緊急分析。

6.1　リー微分（単一政策の構造変革効果）

背景

「ベトナムの染色工場がオフラインになる」というベクトル場 X_{cyber} を定義し、 この単一事象が経営環境多様体全体をどう変形させるかを定量化する。

コード

```python
# ===== ステップ 5：緊急リー微分分析 =====
# ベクトル場：X_cyber = ベトナム工場の停止

def compute_lie_derivative(decoder, z, X_field, epsilon=1e-4):
    """
    点 z におけるベクトル場 X に沿った計量テンソル g のリー微分 L_X(g) を計算する。

     L_X(g)_ij は、X のフローに沿って動いたとき「風景の形」がどう変わるかを捉える。
     L_X(g)の固有値が大きければ、その政策 X は地形を劇的に再形成する。
     """
    dim = len(z)
    g_z = compute_pullback_metric(decoder, z, epsilon)

    # X に沿って小さく流す
    z_flowed = z + epsilon * X_field(z)
    g_flowed = compute_pullback_metric(decoder, z_flowed, epsilon)

    # リー微分 ≈ (g_flowed - g) / epsilon（1 次近似）
    Lie_X_g = (g_flowed - g_z) / epsilon

    return Lie_X_g

# サイバー攻撃ベクトル場を定義
# ベトナムがオフラインになると、以下の指標が変化する：
def X_cyber(z):
    """ベトナム工場停止を表すベクトル場。"""
    dx = np.zeros_like(z)
    # 潜在次元 3 は主に「生産能力」に対応
    # 潜在次元 5 は「サプライチェーンの信頼性」に対応
    # 潜在次元 7 は「コスト構造」に対応
    dx[3] = -0.8   # 生産能力が 80%低下（ベトナム = EU 生産の 40%）
    dx[5] = -0.6   # サプライチェーンの信頼性が急落
    dx[7] = +0.4   # コスト構造が急騰（緊急調達）
    return dx

# 現在の経営状態でリー微分を計算
z_current = vae.encode(X_current_state)
L_cyber = compute_lie_derivative(vae.decoder, z_current, X_cyber)

# リー微分の固有値分析
eigenvalues_lie = np.linalg.eigvalsh(L_cyber)
```

```
print("リー微分の固有値スペクトル：")
print(f" 最大固有値：{eigenvalues_lie[-1]:.4f}")
print(f" 最小固有値：{eigenvalues_lie[0]:.4f}")
print(f" トレース（総変形量）：{np.trace(L_cyber):.4f}")
```

解説

リー微分の定義：

$$(\mathcal{L}_X g)_{ij} = \nabla_i X_j + \nabla_j X_i$$

ベクトル場 X のフロー ϕ_t に沿って計量がどう変化するかを記述する：$\phi_t^* g = g + t(\mathcal{L}_X g) + O(t^2)$。

ケーススタディの結果：

- 最大固有値：3.7142 → 多様体は「コスト」方向に 3.7 倍膨張
- 最小固有値：−2.1893 → 「生産能力」方向に 2.2 倍収縮
- トレース：4.2831 → 経営環境の「体積」が全体として膨張、不確実性が全面的に増大

Killing 場との対比：リー微分が 0 となる場合、ベクトル場 X は Killing 場と呼ばれ、 その政策は「経営環境の構造を保存したまま状態を移動させる」穏やかな政策である。 本ケースでは $\mathcal{L}_X g \neq 0$ であり、**構造変革型**の事象である。 従来の政策評価手法（DID、回帰不連続デザイン）は「結果の差分」を測定するが、 リー微分は「データ空間の幾何構造そのものの変形」を測定する 質的に異なる情報を提供する。

6.2 リー括弧（複数政策の創発効果）

背景

本ケーススタディの **核心的な瞬間**。 X_{cyber} と $X_{\text{trade_policy}}$（エンティティ・リスト施行）のリー括弧 $[X_{\text{cyber}}, X_{\text{trade_policy}}]$ を計算し、 2 つの攻撃が単独では生じない第 3 の効果を検出する。

コード

```
# ===== リー括弧：サイバー × 通商政策の相互作用 =====

def X_trade_policy(z):
    """エンティティ・リスト施行を表すベクトル場。"""
    dx = np.zeros_like(z)
    # 潜在次元 1 は「サプライチェーンの調達選択肢」に対応
    # 潜在次元 6 は「規制コンプライアンスコスト」に対応
    dx[1] = -0.5   # 調達選択肢が狭まる（ブラックリスト入りしたサプライヤー）
    dx[6] = +0.7   # コンプライアンスコストが急増
    return dx


def lie_bracket(X, Y, z, epsilon=1e-4):
    """
    [X, Y](z) = ベクトル場 X と Y のリー括弧を計算する。

    [X, Y]は、X も Y も単独では生み出さない効果を捉える。
    これが「干渉パターン」—— 2 つの政策／事象の組み合わせから
    生まれる創発現象だ。
    """
    # [X, Y] = dY/dz · X - dX/dz · Y（ヤコビアンベースの計算）
```

```
    dim = len(z)
    bracket = np.zeros(dim)

    for i in range(dim):
        z_plus = z.copy(); z_plus[i] += epsilon
        z_minus = z.copy(); z_minus[i] -= epsilon

        dY_di = (Y(z_plus) - Y(z_minus)) / (2 * epsilon)
        dX_di = (X(z_plus) - X(z_minus)) / (2 * epsilon)

        bracket += dY_di * X(z)[i] - dX_di * Y(z)[i]

    return bracket


# 創発効果を計算
bracket_result = lie_bracket(X_cyber, X_trade_policy, z_current)
print("リー括弧 [X_cyber, X_trade_policy]：")
print(f" 大きさ：{np.linalg.norm(bracket_result):.4f}")
print(f" 方向：{bracket_result / np.linalg.norm(bracket_result)}")
```

解説

リー括弧の意味：2 つのベクトル場 V_A、V_B のリー括弧 $[V_A, V_B]$ は、「2 つの政策を順番に実行したとき、実行順序によって結果がどれだけずれるか」を定量的に測定する。$[V_A, V_B] = 0$ なら 2 つの政策は干渉しない（順序に依存しない）。$[V_A, V_B] \neq 0$ ならば非可換であり、**創発現象**が生じる。

ケーススタディの結果：

- 大きさ：2.8934 → 強い創発効果
- 主方向：潜在次元 4 と 6（「ブランド評判」と「投資家の信頼」）の負方向

致命的な解釈：

> サイバー攻撃だけなら、生産にダメージを与える。エンティティ・リストだけなら、調達を制限する。 しかし両方が合わさると、どちらか単独では生じない第 3 の効果が生まれる： ブランド評判と投資家の信頼の壊滅的な崩壊。

なぜか? ランサムウェアの実行者が、綾鳥が新疆綿を使い続けている証拠を公開すると脅迫した。 エンティティ・リスト施行と同時にこの証拠が表面化すれば、語られるストーリーは「綾鳥は強制労働の綿花を秘密裏に使い続けていた上に、国家アクターにハッキングされた」となり、**ブランド消滅イベント**を引き起こす。

これがリームをして「身代金は陽動。文書の公開が本当の武器」と判断させ、 取締役会で「**工場復旧より先にブランド防衛**」という直感に反する処方を提示する根拠となった。

7　ステップ 6：MC ドロップアウトによる不確実性推定

7.1　MC ドロップアウト関数

背景

ステップ 1 ―― 5 で計算されたすべての量（曲率、リー微分、最適経路）に 信頼区間を付与する。 本ケーススタディで **原則 3「分かることと分からないことを分けろ」** を実装する中核機構。

コード

```
# ===== ステップ 6：MC ドロップアウトによる不確実性推定 =====
# ドロップアウトを有効にした状態で 200 回の順伝播を実行

def mc_dropout_prediction(model, z_input, n_samples=200):
    """
    MC ドロップアウト：ドロップアウトを ON にして予測し、200 回繰り返す。
    平均 = 最良推定値。分散 = 不確実性。
    """
    model.train() # ドロップアウトを有効に保つ
    predictions = []

    for _ in range(n_samples):
        with torch.no_grad():
            pred = model(z_input)
        predictions.append(pred.numpy())

    predictions = np.array(predictions)
    mean_pred = predictions.mean(axis=0)
    std_pred = predictions.std(axis=0)

    # 95%信頼区間
    ci_lower = mean_pred - 1.96 * std_pred
    ci_upper = mean_pred + 1.96 * std_pred

    return mean_pred, std_pred, ci_lower, ci_upper

# 例：EU 生産能力予測、Day 30
mean, std, ci_lo, ci_hi = mc_dropout_prediction(
    vae_decoder, z_day30_forecast
)

print("EU 生産能力予測（Day 30）：")
print(f" 最良推定値：{mean[production_idx]*100:.1f}%")
print(f" 95%信頼区間：[{ci_lo[production_idx]*100:.1f}%, "
      f"{ci_hi[production_idx]*100:.1f}%]")
print(f" 不確実性（σ）：{std[production_idx]*100:.1f}%")
```

解説

Gal & Ghahramani (2016) の理論：推論時もドロップアウトを ON にして同じ入力に対して T 回（典型的に T = 100 ―― 200）順伝播し、出力の平均を予測値、分散を不確実性とする。 ドロップアウトマスクの集合は ベイズニューラルネットワークの変分近似事後分布に対応する。

Before / After の対比：

- Before（MC ドロップアウトなし）：「来四半期の売上は 120 億円です」 —— 自信は不明
- After（MC ドロップアウトあり）：「来四半期の売上は 120 億円（95% 信頼区間：105 —— 135 億円）です」

ケーススタディでの応用例（Day 30 の EU 生産能力予測）：

- 最良推定値：91.3%
- 95% 信頼区間：[82.7%, 99.9%]
- 不確実性（σ）：4.4%
- キャリブレーション・チェック：93.2%（90 —— 100% の許容範囲内）

キャリブレーションの重要性： モデルが予測する 95% CI が実際に真の値をカバーする割合を測定し、 過信（CI が狭すぎる）も過小評価（CI が広すぎる）もしていないことを検証する。 93.2% は許容範囲内であり、本ケーススタディの不確実性推定は信頼できる。

取締役会への提示： リームは綾鳥の取締役会に対し、点推定ではなく区間推定で報告した：「EU 生産が Day 30 までに 91% に達すると申し上げるとき、 同時にこうも申し上げています —— 83% から 100% の間にある確率が 95% です。 これは大きく外れうる一点予測ではなく、我々が知っていることと知らないことを正直に表す範囲です」。

これが GI 理論の **原則 3：知っていることと知らないことを分けよ** の運用である。

8　ステップ 7：SHAP と DML

本編における **第 5 月**（2026 年 11 月、偽造品危機）の作業。

8.1　SHAP と DML の併用

背景

第二の波 —— **AI 生成偽造品の津波**。フランスのセンチメント・スコアが 72 時間で 340% 急上昇したが、対応するニュースイベントがない。 リンの異常検知から始まったこの危機において、SHAP（要因分解）と DML（因果効果推定）の両方を実行することで、 ブランド低下の支配的要因を特定し、かつ介入効果を定量化する。

コード

```
# 更新された多様体：偽造品インパクト分析
# カテゴリ 5（消費者センチメント）に新指標を追加：
# 'counterfeit_detection_frequency' —— 週あたりの確認済み偽造品報告数

# SHAP 分析（ステップ 7）：ブランド低下の要因は何か？
import shap

explainer = shap.DeepExplainer(model, background_data)
shap_values = explainer.shap_values(X_current)

print("ブランド低下への SHAP 寄与度 TOP 5（2026 年 11 月）：")
print(" 1. counterfeit_detection_frequency: SHAP = -0.847（支配的）")
print(" 2. sns_negative_sentiment_eu:        SHAP = -0.523")
print(" 3. entity_list_association_score:     SHAP = -0.312")
print(" 4. cyber_incident_recency:           SHAP = -0.198")
print(" 5. eu_regulatory_compliance_gap:      SHAP = -0.145")

# DML（因果推論）—— 偽造品は株価下落の原因か？
print("\nDML 因果効果推定：")
print(" 偽造品頻度の株価に対する因果効果：")
print(" ATE = 偽造品報告 100 件あたり -4.2 円/株")
print(" 95% CI：[-5.1 円, -3.3 円]")
print(" → 因果関係を確認（単なる相関ではない）")
```

解説

SHAP（Shapley 値）の意味： ブランド低下の各要因の 相対的寄与度を Shapley 値（協力ゲーム理論からの値）として計算する。

ケーススタディでの結果（TOP 5）：

1. `counterfeit_detection_frequency`: SHAP = -0.847（**支配的**）
2. `sns_negative_sentiment_eu`: SHAP = -0.523
3. `entity_list_association_score`: SHAP = -0.312
4. `cyber_incident_recency`: SHAP = -0.198
5. `eu_regulatory_compliance_gap`: SHAP = -0.145

DML（Double Machine Learning, Chernozhukov et al. 2018）： SHAP は「何が効いているか」（相関）を答えるが、DML は「何が原因で何が結果か」（因果）を答える。

ケーススタディの DML 結果：

- ATE（平均処置効果）：偽造品報告 100 件あたり −4.2 円/株
- 95% CI：[−5.1, −3.3] 円

両者の処方が分かれる場面（リルル先生の解説）：「SHAP 分析の結果が示しているのは『偽造品が最大の問題だ』ということです。 DML 分析の結果は『偽造品を是正すれば、回復への最大の因果効果がある』という気づきです。この場合は両者が一致していますが、DML の結果は SHAP にはないものを与えてくれます——反事実：『偽造品を X だけ減らせば、株価は Y だけ回復する』」。

ケーススタディ A（製造業 X 社）では SHAP 1 位「広告費の増額」と DML 1 位「R&D 投資の拡大」が乖離したが、 本ケースでは両者が一致した。「両者が一致するか乖離するか自体が、施策の信頼性を測る情報である」という観点も重要。

9　ステップ 8：多様体上の最適制御

本編における **第 2 月**（2026 年 10 月）—— サイバー攻撃直後の **48 時間以内** に実行された緊急最適制御計算。

9.1　ManifoldOptimalControl クラス

背景

リー括弧分析でブランド評判への致命的な打撃が予測された後、それを防ぐ最適経路をポントリャーギンの最大値原理によって求める。

コード

```
# ===== ステップ 8：緊急最適制御 =====
# 目的：現在の状態から「EU 生産復旧、ブランド評判保全」まで
#       最短時間で移動する
# 制約：ベトナム 14 日以上オフライン、エンティティ・リスト有効、
#       緊急措置の予算制約

import torchdiffeq


class ManifoldOptimalControl:
    """
    経営環境多様体上のポントリャーギンの最大値原理。

    状態：z ∈ M（8 次元潜在多様体）
    制御：u ∈ U（利用可能な経営政策の集合）
    ダイナミクス：dz/dt = f(z, u) + クリストッフェル補正
    目的関数：min ∫ L(z, u) dt + Φ(z(T))（コスト＋終端ペナルティの最小化）
    """

    def __init__(self, decoder, metric, christoffel):
        self.decoder = decoder
        self.metric = metric
        self.christoffel = christoffel

    def hamiltonian(self, z, p, u):
        """
        H(z, p, u) = p · f(z, u) - L(z, u)

        p：随伴変数（共状態）—— 各状態次元の「影の価格」
        u：制御入力 —— 評価対象の経営政策
        """
        f_zu = self.dynamics(z, u)
        L_zu = self.running_cost(z, u)
        return np.dot(p, f_zu) - L_zu

    def dynamics(self, z, u):
        """
        多様体上の経営ダイナミクス。
        dz/dt = f(z, u) - Γ^k_ij dz^i/dt dz^j/dt（クリストッフェル補正）

        クリストッフェル補正は極めて重要 —— 空間の曲率を考慮する。
        これがなければ、最適経路は空間が平坦であるかのように計算され、
        自然な曲率に「逆らう」政策になってしまう。
```

```
        """
        # Neural ODE からの基本ダイナミクス
        f_base = self.neural_ode(z, u)

        # 曲がった空間のためのクリストッフェル補正
        christoffel_correction = np.zeros_like(z)
        for k in range(len(z)):
            for i in range(len(z)):
                for j in range(len(z)):
                    christoffel_correction[k] -= (
                        self.christoffel[k, i, j] * f_base[i] * f_base[j]
                    )

        return f_base + christoffel_correction
```

解説

ハミルトニアン定式化：

$$H(z, p, u) = p \cdot f(z, u) - L(z, u)$$

ここで z は状態（潜在多様体上の位置）、p は随伴変数（共状態、各状態次元の「影の価格」）、u は制御入力。

クリストッフェル補正の決定的重要性：

$$\frac{dz^k}{dt} = f^k(z, u) - \Gamma^k_{ij}(z) \frac{dz^i}{dt} \frac{dz^j}{dt}$$

これが GI 理論の核心である。クリストッフェル補正がなければ、最適経路は空間が平坦であるかのように計算され、**自然な曲率に「逆らう」政策**になってしまう。これが本編における「測地線は直感が示す方向を必ずしも向いていません」というリルル先生の警告の数学的内容。

Neural ODE との関連：基本ダイナミクス `f_base = self.neural_ode(z, u)` は、ステップ 1 で訓練した Neural ODE を参照する。Chen et al. (2018) の随伴法は ポントリャーギンの随伴方程式の離散アナログであり、両者の数学的対応が「学習」と「最適制御」を統合する鍵である。

9.2　制御オプションの定義と最適経路求解

背景

経営陣が即座に実行可能な 7 つの緊急対応オプション（A —— G）を辞書で定義し、ポントリャーギンの最大値原理に渡す。出力される最適経路は本ケーススタディで取締役会承認を得た 98 億円の緊急対応プランそのものである。

コード

```
# 利用可能な制御入力（緊急経営政策）
control_options = {
    'A_reroute_turkey': {
        'description': 'EU 生産をトルコ（イスタンブール）工場に振り替え',
        'capacity': 0.25,   # ベトナムの EU 生産の 25%を吸収可能
        'lead_time': 7,     # 立ち上げに 7 日
        'cost': 2.8,        # 月あたり 28 億円の追加コスト
    },
```

```
    'B_reroute_italy': {
        'description': 'EU 生産をイタリア（コモ）工場に振り替え',
        'capacity': 0.15,   # ベトナムの EU 生産の 15%を吸収可能
        'lead_time': 3,     # 3 日（EU 生産の設備は整っている）
        'cost': 4.2,        # 42 億円（イタリアの人件費が高い）
    },
    'C_reroute_japan': {
        'description': '生産を日本（新潟）工場に振り替え',
        'capacity': 0.10,   # 輸出生産の能力は限定的
        'lead_time': 14,    # 14 日（設備の再構成が必要）
        'cost': 5.1,        # 51 億円（最もコストが高い選択肢）
    },
    'D_emergency_outsource': {
        'description': 'バングラデシュの契約サードパーティへの緊急外注',
        'capacity': 0.20,   # 20%の能力が利用可能
        'lead_time': 21,    # 21 日（品質検証が必要）
        'cost': 1.5,        # 15 億円（最安だがリスクが最も高い）
    },
    'E_brand_defense': {
        'description': '能動的ブランド防衛：記者会見＋透明性キャンペーン',
        'capacity': None,   # 該当なし（ブランド関連、生産ではない）
        'lead_time': 2,     # 48 時間の準備
        'cost': 0.3,        # 3 億円（PR 会社＋メディア購入）
        'reputational_impact': +0.35,  # ブランド評判ダメージを 35%低減
    },
    'F_cyber_response': {
        'description': 'サイバーセキュリティ・インシデント対応＋フォレンジック調査',
        'lead_time': 1,     # 即時
        'cost': 0.8,        # 8 億円（CIRT チーム＋外部フォレンジック）
    },
    'G_legal_counter': {
        'description': '新疆綿段階的廃止を証明する先制法的措置',
        'lead_time': 5,     # 5 日（法務準備）
        'cost': 0.2,        # 2 億円（弁護士費用）
        'reputational_impact': +0.50,  # エンティティ・リスト関連の
                                       # ブランド評判ダメージを大幅に低減
    },
}

# 最適制御問題を解く
# 以下を最小化する制御の組み合わせとタイミングを求める：
#   (1) 総コスト
#   (2) EU 生産能力 80%復旧までの時間
#   (3) ブランド評判ダメージ（リー括弧の大きさで測定）

optimal_trajectory = solve_pontryagin(
    initial_state=z_current,
    target_state=z_target,  # 「EU 生産 80%以上復旧、ブランド評判安定」
    controls=control_options,
    time_horizon=90,        # 90 日の計画期間
    manifold_metric=pullback_metric,
    christoffel_symbols=christoffel,
)

print("最適緊急対応プラン：")
for day, actions in optimal_trajectory.schedule.items():
    print(f" Day {day}：{', '.join(actions)}")
print(f"\n 総コスト：{optimal_trajectory.total_cost:.1f}十億円")
```

```
print(f"EU 生産能力 80%復旧：Day {optimal_trajectory.recovery_day}")
print(f"ブランド評判ダメージ（リー括弧残余）：{optimal_trajectory.rep_damage:.2f}")
```

解説

7 つの制御オプション：

- A：トルコ工場への振り替え（25% 容量、7 日、28 億円/月）
- B：イタリア工場への振り替え（15% 容量、3 日、42 億円）
- C：日本工場への振り替え（10% 容量、14 日、51 億円）
- D：バングラデシュ緊急外注（20% 容量、21 日、15 億円）
- E：ブランド防衛（PR キャンペーン、2 日、3 億円、評判ダメージ -35%)
- F：サイバー対応（即時、8 億円）
- G：先制法的措置（5 日、2 億円、評判ダメージ -50%)

最適制御の出力（Day 0 —— 30）：

- Day 0：F（即時）、E（準備開始）
- Day 2：E（記者会見実施）
- Day 5：G（米国商務省への先制的情報開示）
- Day 7：A（トルコ立ち上げ）
- Day 14：ベトナム工場部分復旧
- Day 30：ベトナム工場全面復旧

総コスト 98 億円、EU 生産能力 80% 復旧 Day 12、評判ダメージ $2.89 \to 0.31$。

核心的な洞察：最適経路は生産復旧から始まらない。ブランド評判の防衛から始まる。 **なぜか?** プルバック計量が示すところでは、「ブランド評判」次元が現在位置の近傍で **最も急な勾配**を持っている。測地線（曲がった空間を通る最小抵抗の経路）は、 ブランド価値の保護を最初に行うことで「下り坂」を作り、後続の生産復旧をより容易・安価にする。

これは従来の危機管理の直感（工場復旧最優先）に反するが、多様体の幾何学が示す処方である。

業界転用の注意：本ケースでは制御オプションが製造業に固有（工場振り替え、PR 対応など）であり、 他業界に適用する場合は制御オプションの再設計が必要。 ただし、「最適経路が直感に反する場合がある」という構造的特徴は業界によらず保持される。

10　幾何学的拡張

本ケーススタディで使用された GI 理論第 13 章 —— 第 28 章の幾何学的拡張機能の実装。

10.1　層理論による偽造品検知

背景

第 5 月の偽造品危機への対応として配備されたシステム。 本物の AYATORI 製品は複数のデータソース（製造記録、出荷マニフェスト、小売在庫、顧客レビュー）にわたって 整合的なシグネチャを持つが、偽造品は不整合を生む（異なるデータソースが連携していないため）。

コード

```
# ===== 層ベースの偽造品検知（第 18 章）=====

# 核心的な洞察：正規の AYATORI 製品は複数のデータソース
# （製造記録、出荷マニフェスト、小売在庫、顧客レビュー）に
# わたる「整合性のシグネチャ」を持つ。偽造品には不整合が

# 生じる —— 異なるデータソースが連携していないため。

# モデル：サプライチェーングラフ上のセル層（cellular sheaf）
# 各ノード：サプライチェーンの段階（工場 → 倉庫 → 小売 → 顧客）
# 各辺：段階間の移転
# 層データ：各ノードにおける製品属性

# 層のラプラシアンが不整合を検知する：
# L_sheaf · x ≠ 0 ならば、データに不整合が存在する
# L_sheaf · x の大きさが不整合の程度を示す

def detect_counterfeit_via_sheaf(product_id, supply_chain_data):
    """
    層コホモロジーを用いた偽造品検知。

    正規品はサプライチェーンの全ノードにわたって整合的なデータを持つ。
    偽造品には不整合がある —— 工場の記録が出荷マニフェストと一致しない、
    あるいは小売在庫が製造ロットコードと一致しない。
    """
    # サプライチェーングラフ上に層を構成
    sheaf = construct_supply_chain_sheaf(supply_chain_data)

    # 層のラプラシアンを計算
    L = sheaf.laplacian()

    # 整合性をチェック
    x = sheaf.get_product_data(product_id)
    inconsistency = np.linalg.norm(L @ x)

    if inconsistency > threshold:
        return f"偽造品検知：不整合スコア = {inconsistency:.2f}"
    else:
        return f"正規品：整合スコア = {1.0 - inconsistency:.2f}"

# 検知例：
# 製品 AYT-2026-FR-00847（パリのポップアップで販売）：
```

```
#   製造記録：新潟工場、ロット NI-2026-0847
#   出荷マニフェスト：日本からの該当する輸出記録なし
#   → 不整合：製品は日本製を主張しているが、日本から出荷されていない
#   → 偽造品検知：不整合スコア = 4.73
```

解説

セル層（cellular sheaf）：サプライチェーンを以下のように層構造としてモデル化する。

- ノード：サプライチェーンの段階（工場 → 倉庫 → 小売 → 顧客）
- 辺：段階間の移転
- 層データ：各ノードにおける製品属性

層ラプラシアンの不整合検知原理：

$$\|L_{\text{sheaf}} \cdot x\| > \theta \implies \text{不整合あり}$$

ここで L_{sheaf} はサプライチェーン上の制限写像から構成される層ラプラシアン。

ケーススタディでの検知例：製品 AYT-2026-FR-00847（パリのポップアップで販売）

- 製造記録：新潟工場、ロット NI-2026-0847
- 出荷マニフェスト：日本からの該当する輸出記録なし
- 不整合スコア：4.73（閾値 2.0 を大きく超過）→ **偽造品**

第 8 章での再利用：本コードは偽造品検知のために設計されたが、 同じシステムが**第 8 章の内部脅威検知**（武田の漏洩）でも層ラプラシアンの不整合スコア 6.73 を検出した。 単一の異常検知エンジンが、外部脅威と内部脅威の両方を捉えたことが本ケースの教訓の一つ。

10.2　ベクトル場（DPP 規制 / 敵対的買収）

背景

DPP 規制（第 5 月）と敵対的買収（第 6 —— 7 月）をそれぞれベクトル場としてモデル化し、 既存のサイバー・通商政策ベクトル場との **多方向リー括弧**を計算する。

コード（DPP 規制）

```python
# 新指標：eu_dpp_compliance_gap（0 = 完全準拠、100 = 完全非準拠）
# 綾鳥の現在のギャップ：100 点満点中 73

# 規制ベクトル場のリー微分
def X_dpp_regulation(z):
    """DPP 施行を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[1] = -0.3   # サプライチェーンの調達選択肢が狭まる（完全なトレーサビリティが必要）
    dx[6] = +0.9   # 規制コンプライアンスコストが急増
    dx[2] = -0.4   # 売上が脅かされる（非準拠製品は EU から締め出し）
    return dx

L_dpp = compute_lie_derivative(vae.decoder, z_current, X_dpp_regulation)
print(f"DPP 規制インパクト —— リー微分のトレース：{np.trace(L_dpp):.2f}")
print(f"リスクにさらされている推定売上：{22 * 0.73:.1f}十億円")
print(f"  （EU 起源の総売上 22% × 非準拠率 73%）")
```

コード（敵対的買収）

```
# ===== 敵対的買収：多様体緊急更新 =====

# 新たな外部ショックのベクトル場
def X_takeover(z):
    """敵対的買収圧力を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[0] = +0.2   # TOB プレミアムにより一時的に売上が押し上げ
    dx[4] = -0.8   # 戦略的自律性が崩壊（買収者が取締役会を支配）
    dx[6] = +0.3   # コンプライアンスコストが上昇（統合の混乱）
    dx[7] = +0.5   # 全次元にわたる不確実性の急増
    return dx

# 4 方向リー括弧分析
# [[[X_cyber, X_trade], X_dpp], X_takeover]
quad_bracket = compute_nested_lie_bracket(
    [X_cyber, X_trade_policy, X_dpp_regulation, X_takeover], z_current
)
print(f"4 方向リー括弧の大きさ：{np.linalg.norm(quad_bracket):.2f}")
```

解説

3 方向リー括弧 $[X_{\text{cyber}}, X_{\text{trade}}, X_{\text{dpp}}]$：DPP は透明性を要求するが、エンティティ・リストは可視性を罰する。同時に透明かつ不可視であることはできない。これは規制空間の幾何学に埋め込まれた**位相的な罠**。

倉田教授の代数的位相幾何学的診断：「これは非自明な第一コホモロジー類だ。大域的な解を見つける障害は数値的なものではない —— 構造的なもの。どれだけの資金や努力を投じても、真正の位相的障害を解消することはできない」。これが手術理論の介入を必要とする根拠となる（次節）。

4 方向リー括弧 $[[[X_{\text{cyber}}, X_{\text{trade}}], X_{\text{dpp}}], X_{\text{takeover}}]$：大きさ 7.41 —— これまで測定された中で最も強い創発効果。他の 3 つの危機がアクティブな状態でメゾン・オロールが綾鳥を買収すれば、統合は壊滅的に不安定になる（イノベーション・パイプラインの喪失）。

10.3　手術理論：サプライチェーン再構築

背景

第 5 月、DPP 規制の前倒しで生じた「位相的罠」を、手術理論で多様体の位相そのものを変えることによって解消する。

コード

```
# ===== 第 13 章：手術理論の適用 =====
# 問題：DPP の透明性とエンティティ・リストの隠蔽の間の位相的障害
# 解決策：新疆に隣接するサプライチェーン・セグメントを完全に「切除」し、
#         完全に透明で DPP 準拠の代替物を「接着」する

# 手術理論：2 ステップ
# ステップ 1：切除 —— 多様体の問題領域を取り除く
# ステップ 2：接着 —— より良い位相を持つ代替領域を取り付ける

# 「問題領域」とは、新疆に隣接する仲介業者を経由する
```

```
# サプライチェーンパスの集合である。
# これらのパスが位相的障害を生み出している。

# 手術の処方：
surgery_plan = {
    'cut': [
        '仲介業者 A（シンガポール）—— 綿花供給の 8%、新疆に関連',
        '仲介業者 B（香港）—— 綿花供給の 5%、エンティティ・リストのリスク',
        '仲介業者 C（ベトナム）—— 綿花供給の 3%、出所不明',
    ],
    'glue': [
        'エジプトの綿花協同組合との直接契約（GOTS 認証済み）',
        'インドのオーガニック綿花農場集団との直接契約（BCI 認証済み）',
        '米国のピマ綿花サプライヤーとの直接契約（DPP 完全トレーサブル）',
    ],
    'verification': 'コボルディズム条件：切除領域の境界は接着領域の境界と'
                    '一致しなければならない。ビジネスの言葉では：'
                    '代替サプライヤーは、取り除かれた仲介業者と同じ素材'
                    '仕様（繊維長、引張強度、染色堅牢度）を提供しなければならない。',
}

# コボルディズム条件の検証
print("コボルディズム条件チェック：")
print(" 切除境界：綿花供給の 16%、仕様：Gossypium hirsutum、")
print("   繊維長 28-32mm、等級：Shankar-6 相当")
print(" 接着境界：エジプト GOTS（29mm）+ インド BCI（30mm）+ 米国ピマ（34mm）")
print(" → 仕様は許容範囲内で一致（繊維長±3mm）")
print(" → コボルディズム条件：充足 ✓")
print("")
print(" 手術後の多様体の位相：")
print(" → 第一コホモロジー障害：消滅")
print(" → DPP 準拠パスとエンティティ・リスト回避パスが両立可能に")
print(" → 鞍点が尾根に変わる —— 標準的な測地線制御で航行可能")
```

解説

手術理論の 2 ステップ：

1. **切除**：問題のある領域を取り除く（新疆に隣接する仲介業者 3 社の切り離し、綿花供給の 16%）
2. **接着**：より良い位相を持つ代替領域を取り付ける（エジプト GOTS、インド BCI、米国ピマ、計 16%）

コボルディズム条件（カート＝レイマー、トム）：切除領域の境界は接着領域の境界と一致しなければならない。 ビジネスの言葉では「代替サプライヤーは、取り除かれた仲介業者と同じ素材仕様（繊維長、引張強度、染色堅牢度）を提供しなければならない」。

ケーススタディでの境界条件チェック：

- 切除境界：仕様 Gossypium hirsutum、繊維長 28 —— 32mm、Shankar-6 等級
- 接着境界：エジプト GOTS（29mm）+ インド BCI（30mm）+ 米国ピマ（34mm）
- 結果：許容範囲内で一致（繊維長 ±3mm）—— **コボルディズム条件 充足** ✓

位相的成果：手術後、 第一コホモロジー障害が消滅、DPP 準拠パスとエンティティ・リスト回避パスが両立可能、鞍点が尾根に変わる。 コスト 45 億円、4 ヶ月の実行期間、しかし**機能することの数学的証明**が綾鳥取締役会のコミットメントを引き出した。

10.4　TDA：パーシステント・ホモロジー

背景

メゾン・オロールの株式持株蓄積パターンに対する TDA 適用。リンの観察から、 シェル会社による隠れた持株を金融取引パターンの「形」から検出する。

コード

```
# ===== TDA：金融取引パターンのパーシステント・ホモロジー =====

from ripser import ripser
from persim import plot_diagrams

# メゾン・オロールの株式購入パターンを分析
# データ：日次の購入量、デリバティブポジション、ブローカーの身元

purchase_data = load_financial_data('maison_aurore_stake_buildup.csv')
result = ripser(purchase_data, maxdim=1)

# パーシステンス図が明らかにしたこと：
# - H0（連結成分）：4 つの購入活動のクラスター
#   → 購入活動は 4 つの別々のソースから来ていた

# - H1（ループ）：1 つの持続的なループ
#   → 4 つのソースが循環的パターンで接続されている
#   → これは多層シェル会社構造の特徴

print("TDA パーシステント・ホモロジー結果：")
print(f" H0（連結成分）：4（有意性：{result['dgms'][0][-1][1] - result['dgms'][0][-1][0]:.3f}）")
print(f" H1（ループ）：1 つの持続的ループ（持続性：{result['dgms'][1][0][1] -
↪  result['dgms'][1][0][0]:.3f}）")
print()
print("解釈：")
print(" 持株は 4 つの別々のエンティティを通じて蓄積された。")
print(" それらは循環的な所有構造で接続されている。")
print(" これは、実質的支配者（UBO）を隠蔽するために設計された")
print(" 多層シェル会社構造と整合的である。")
```

解説

パーシステンス図の解釈：

- H_0（連結成分）：4 → 購入活動は 4 つの別々のソースから
- H_1（ループ）：1 つの持続的ループ → 4 つのソースが循環的所有構造で接続

シェル会社のシグネチャ：開示閾値（5%）を超えないようにエンティティを分割し、 それらが循環的に所有しあう構造は、多層シェル会社の特徴的な位相を示す。 統計的手法（相関分析、回帰分析）はこのパターンを見逃すが、**TDA は「データの形そのもの」を見るため**捉えられる。

法執行への活用：パーシステント・ホモロジーで「どんな形を探すべきか」を特定した後、商業データベース（Orbis、Sayari、Dow Jones RiskCenter）でその形を持つ実体を検索する。 機密情報源を保護しながらオープンソース調査で結論を独立に立証するワークフロー。本ケースでは AMF / EU DG FISMA / OFAC の 3 法域同時提出につながった。

10.5　ゲージ変換：多様体の再ランダム化

背景

第 8 月、内部漏洩者（武田）と APT 攻撃でモデルが部分的に流出した後の防衛策。 多様体の幾何学は座標不変だが、潜在ベクトルの数値は座標系の選び方で変わるという性質を利用する。

コード

```python
# ===== 緊急ゲージ変換 =====
# VAE の潜在空間を回転させ、盗まれたデータを無効化する

def gauge_transform_vae(vae_model, rotation_matrix):
    """

    VAE の潜在空間にゲージ変換を適用する。

    核心的な洞察：多様体の幾何学は座標不変である。
    曲率、測地線、リー微分 —— すべての幾何学的量は
    どの座標系でも同じ。しかし潜在ベクトルの数値は
    回転の下で変化する。

    したがって：エンコーダの出力とデコーダの入力を R で回転する。
    下流のすべての幾何学的計算は正しいまま。
    しかし盗まれた潜在ベクトルはゴミになる。
    """
    R = torch.tensor(rotation_matrix, dtype=torch.float32)

    # エンコーダを修正：z_new = R @ z_old
    original_encode = vae_model.encode
    def rotated_encode(x):
        z = original_encode(x)
        return z @ R.T # 回転を適用
    vae_model.encode = rotated_encode

    # デコーダを修正：x_reconstructed = decode(R^T @ z_new) = decode(z_old)
    original_decode = vae_model.decode
    def rotated_decode(z):
        return original_decode(z @ R) # デコード前に回転を元に戻す
    vae_model.decode = rotated_decode

    return vae_model

# ランダムな直交回転行列を生成
from scipy.stats import special_ortho_group
```

解説

核心的洞察：曲率、測地線、リー微分はすべて **テンソル量**であり座標変換に対して不変。 特殊直交群 $SO(8)$ からランダムな回転行列 R を引いて VAE 潜在空間を回転させると、

- 下流のすべての幾何学的計算は正しいまま（テンソル変換則を経由）
- 敵が盗んだ潜在ベクトルは無意味なノイズになる

リームの判断：**48 時間ごとに座標系を回転** —— 敵のインテリジェンスの半減期を 2 日間に強制する。

GI 理論との接続：これは GI 理論第 11 章のワイル幾何（ゲージ不変性）を 防衛戦術として転用した本ケースの独自の貢献的場面。 通貨単位変更不変性の理論が、サイバー防衛のメカニズムに応用される。

10.6　応力テンソル分析：レジリエンス測定

背景

第 10 月、**台風マワル＋パンデミック HCoV-27** の二重ショック直後の分析。 過去の手術理論的介入（サプライチェーン再構築）が、無関係な危機（自然災害+パンデミック）に対する レジリエンスを生んでいたかを測定する。

コード

```
# ===== 第 10 月：自然災害＋パンデミックの影響 =====

# 多様体の緊急更新
z_current = vae.encode(X_latest_data)
R_scalar = compute_scalar_curvature(vae.decoder, z_current)

print(f"台風＋パンデミック後のスカラー曲率：{R_scalar:.2f}")
print(f"災害前の曲率：{-3.4:.2f}")
print()

# 応力テンソル分析（続編第 5 章：「テンソル場」）
def compute_stress_tensor(decoder, z, shock_vector):
    """
    外部ショック下の点 z における応力-エネルギーテンソルを計算する。

    多様体のどこに応力が集中するかを示す。
    脆弱な企業：応力が一点に集中 → 破断。
    強靭な企業：応力が均等に分散 → 弾性変形。
    """
    g = compute_pullback_metric(decoder, z)
    L_shock = compute_lie_derivative(decoder, z, lambda z: shock_vector)

    # 応力テンソル：T_ij = g_ik * L^k_j
    g_inv = np.linalg.inv(g)
    T = g_inv @ L_shock

    # 固有値分解：主応力と主方向
    eigenvalues, eigenvectors = np.linalg.eigh(T)

    return eigenvalues, eigenvectors

# 複合ショックベクトル：台風＋パンデミック
shock = np.zeros(8)
shock[3] = -0.95   # ベトナム生産能力 → ほぼゼロ
shock[5] = -0.70   # サプライチェーン信頼性 → 深刻な混乱
shock[7] = +0.60   # コスト構造 → 緊急価格
shock[1] = -0.40   # 調達選択肢 → 検疫制限

eigenvalues, eigenvectors = compute_stress_tensor(vae.decoder, z_current,
 shock)

print("応力テンソル分析（テンソル場分析）：")
print(f" 最大主応力：{eigenvalues[-1]:.2f}")
```

```
print(f"    方向：主に「ベトナム生産」次元")
print(f" 最小主応力：{eigenvalues[0]:.2f}")
print(f"    方向：主に「日本国内」次元")
print()
print("レジリエンス評価：")
stress_concentration = eigenvalues[-1] / np.mean(np.abs(eigenvalues))
if stress_concentration > 3.0:
    print(f" 応力集中比：{stress_concentration:.1f}")
    print(f" → 警告：応力が集中しており、分散していない")
    print(f" → 介入なしでは、破断リスクが高い")
else:
    print(f" 応力集中比：{stress_concentration:.1f}")
    print(f" → 応力は合理的に分散している。弾性変形の可能性が高い。")
```

解説

応力テンソル T_{ij}：

$$T_{ij} = g_{ik} \cdot \mathcal{L}^k_j$$

固有値分解により主応力（最大固有値）と主方向（対応する固有ベクトル）を求める。

ケーススタディの結果：

- 最大主応力：4.87（方向：「ベトナム生産」次元)
- 最小主応力：0.23（方向：「日本国内」次元)
- **応力集中比：2.7**（4.87 / 平均絶対値）

レジリエンス評価：応力集中比 < 3.0 → 応力は合理的に分散、弾性変形の可能性が高い。

リルル先生の核心的解説：「6 ヶ月前のサプライチェーン手術前なら、この数字は 5.0 を超えていました。ベトナムが EU 生産の 40% の単一障害点だったからです。手術理論的介入と振り替え能力構築が、複数の荷重支持構造にストレスを分散させる効果を生み出しています」。

転用可能なレジリエンス： ユキの問い：「前の危機を生き延びるためにやったことが、この危機に対して綾鳥をより強くしたんですか? まったく種類の違う問題なのに?」 リルル先生の答え：「危機の種類を超えて転用可能な構造的レジリエンスを構築します。多様体は危機にもかかわらず頑健になるのではなく、危機への対応として行われた幾何学的再構築によって頑健になるのです」。

これが **古今東西 7 つの文明の古典**（チャーチル、二宮尊徳、カミュ、司馬遷、アイスキュロス、セネカ、クルアーン、ルーミー、バガヴァッド・ギーター、詩篇 121 篇）が独立に到達した「**苦難は強さに転化しうる**」という洞察を、ついに数式で定量化した瞬間である。

11　長期シミュレーション：シンプレクティック幾何学

本編における **第 10 月**の作業。 CEO 西村の問い「5 年後の綾鳥はどうなっていますか?」への回答。

11.1　Störmer-Verlet シンプレクティック積分器

背景

5 年間（60 ヶ月）の経営軌道シミュレーションでは、標準的数値積分器（Euler 法、Runge-Kutta 法）の エネルギードリフトが致命的になる。シンプレクティック積分器（位相空間体積を厳密保存）が必要。

コード

```
# ===== 第 15 章：長期シミュレーションのためのシンプレクティック幾何学 =====

# なぜシンプレクティックか？

# 標準的な数値積分器（オイラー法、ルンゲ＝クッタ法）は
# 長期シミュレーションでエネルギーのドリフトが蓄積する。
# 5 年のシミュレーション時間の後、軌道は真の経路から
# 大きく逸れている可能性がある。
#
# シンプレクティック積分器（シュテルマー＝ヴェルレ法）は
# シンプレクティック構造 —— 「位相空間内の体積」—— を
# 厳密に保存する。つまりエネルギー（企業価値として解釈）は
# 非常に長いシミュレーションにわたっても機械精度で保存される。

def stormer_verlet_on_manifold(z0, p0, H, christoffel, dt, n_steps):
    """
    経営環境多様体上のシュテルマー＝ヴェルレ・シンプレクティック積分器。

    z：多様体上の位置（経営状態）
    p：運動量（経営状態の変化率）
    H：ハミルトニアン（企業価値関数）
    christoffel：クリストッフェル記号（曲率補正）

    クリストッフェル補正は不可欠 —— なければ積分器は
    空間を平坦として扱い、曲率の効果を完全に見落とす。
    """
    z = z0.copy()
    p = p0.copy()
    trajectory = [(z.copy(), p.copy())]

    for step in range(n_steps):
        # ステップ 1：p を半ステップ進める（クリストッフェル補正付き）
        dH_dz = compute_gradient(H, z)
        christoffel_term = np.zeros_like(z)
        for k in range(len(z)):
            for i in range(len(z)):
                for j in range(len(z)):
                    christoffel_term[k] += christoffel[k,i,j] * p[i] * p[j]
        p = p - 0.5 * dt * (dH_dz + christoffel_term)

        # ステップ 2：z を 1 ステップ進める
        g_inv = np.linalg.inv(compute_pullback_metric(vae.decoder, z))
```

```
        z = z + dt * (g_inv @ p)

        # ステップ 3：p を残りの半ステップ進める
        dH_dz = compute_gradient(H, z)
        christoffel_term = np.zeros_like(z)

        for k in range(len(z)):
            for i in range(len(z)):
                for j in range(len(z)):
                    christoffel_term[k] += christoffel[k,i,j] * p[i] * p[j]
        p = p - 0.5 * dt * (dH_dz + christoffel_term)

        trajectory.append((z.copy(), p.copy()))

    return trajectory

# 5 年間のシミュレーション：60 ヶ月、月次ステップ
trajectory_5yr = stormer_verlet_on_manifold(
    z0=z_current,
    p0=p_current, # 現在の「速度」（経営状態の変化率）
    H=enterprise_value_hamiltonian,
    christoffel=christoffel_symbols,
    dt=1.0, # 1 ステップあたり 1 ヶ月
    n_steps=60 # 60 ヶ月 = 5 年
)

# 軌道に沿った主要指標の抽出
for month, (z, p) in enumerate(trajectory_5yr[::12]):  # 年次スナップショット
    year = 2027 + month // 12
    decoded = vae.decode(z)
    revenue = decoded[0] * scaler.scale_[0] + scaler.center_[0]
    margin = decoded[5] * scaler.scale_[5] + scaler.center_[5]
    print(f" {year}年：売上 {revenue:.0f}十億円、営業利益率 {margin:.1f}%")

# エネルギー保存のチェック（シンプレクティック特性）
H_initial = enterprise_value_hamiltonian(
    trajectory_5yr[0][0], trajectory_5yr[0][1]
)
H_final = enterprise_value_hamiltonian(
    trajectory_5yr[-1][0], trajectory_5yr[-1][1]
)
print(f"\n エネルギー保存：ΔH/H = "
      f"{abs(H_final - H_initial)/abs(H_initial):.2e}")
print(f"  （シンプレクティック積分器はエネルギーを"
      f"{abs(H_final - H_initial)/abs(H_initial)*100:.4f}%の精度で保存） ")
```

解説

シンプレクティック構造の保存：標準的積分器は時間が経つにつれて 「企業価値（ハミルトニアン）」のドリフトを生じさせ、5 年後には予測経路が真の経路から大きく逸れる。Störmer-Verlet 法は**各ステップでシンプレクティック 2-形式** $\omega = dp \wedge dz$ **を厳密に保存**し、エネルギーを機械精度で保存する。

クリストッフェル項：シンプレクティック特性を多様体上で実現するためには $\Gamma^k_{ij} p^i p^j$ 補正項が不可欠。これがなければ積分器は空間を平坦として扱い、 曲率の効果を完全に見落とす。

ケーススタディの 5 年予測（最適制御シナリオ）：

- 2027 年：売上 2,850 億円、営業利益率 8.2%（縮小）
- 2028 年：売上 3,120 億円、10.1%（回復）
- 2029 年：売上 3,480 億円、11.8%（DPP 準拠優位性）
- 2030 年：売上 3,910 億円、13.2%（インド展開）
- 2031 年：売上 4,270 億円、14.1%（定常状態）
- 2032 年：売上 4,580 億円、14.5%（市場リーダー）

反事実（介入なし）：

- 2028 年に営業損失突入、2029 年に死のスパイラル、2030 年に独立企業として存続不可能

エネルギー保存の検証：$\Delta H/H = 3.21 \times 10^{-8}$ —— シンプレクティック積分器は 5 年シミュレーションを通じて 0.0000032% の精度でエネルギーを保存。 これが 60 ヶ月先までの予測の数学的健全性を担保する。

12　14 体 AI エージェント配備

本編における **第 11 —— 12 月**（2027 年 7 —— 8 月）の作業。 GI 理論第 7 章の 3 軸分類に基づくエージェント設計の運用配備。

12.1　エージェント・ロスター

背景

経営環境多様体をデジタルツインとして稼働させ、その上で 14 体の AI エージェントが 深層強化学習により稼働する。本編で議論された二層多様体構造（GI 理論 Vol.2）を実装する具体形。

コード

```
# ===== 第 7 章：綾鳥向け 14 体 AI エージェントの配備 =====

# 3 軸に沿った分類（第 7 章）：
# 軸 1：身体化（物理センサーデータ） vs 非身体化（市場・規制データ）
# 軸 2：反応型（イベントに対応） vs 予見型（イベントを予測）
# 軸 3：単一ドメイン vs クロスドメイン

agent_roster = {
    # === 身体化エージェント（工場・物理系）===
    'Agent_01_Vietnam_Factory': {
        'type': '身体化',
        'behavior': '反応型＋予見型',
        'domain': '生産（ベトナム）',
        'inputs': ['ICS/SCADA センサーデータ', 'エネルギー消費',
                   '不良率', '作業員出勤状況'],
        'function': '工場の健全性監視、設備故障予測、'
                    '異常な ICS 挙動の検知（サイバー防御）',
    },
    'Agent_02_Turkey_Factory': {
        'type': '身体化',
        'behavior': '反応型＋予見型',
        'domain': '生産（トルコ）',
        'inputs': ['Agent_01 と同様、トルコ固有'],
        'function': 'トルコ工場の監視、サージ容量の管理',
    },
    'Agent_03_Italy_Factory': {
        'type': '身体化',
        'behavior': '予見型',
        'domain': '生産（イタリア・コモ）',
        'inputs': ['高級生地品質メトリクス', '職人スケジューリング'],
        'function': 'ラグジュアリー製品品質の最適化、季節需要予測',

},
'Agent_04_Japan_Factory': {
    'type': '身体化',
    'behavior': '反応型',
    'domain': '生産（日本）',
    'inputs': ['新潟・岡山のセンサーデータ'],
    'function': '国内生産の最適化',
},

# === 非身体化エージェント（市場・規制・財務系）===
```

```
'Agent_05_Market_Sentinel': {
    'type': '非身体化',
    'behavior': '予見型',
    'domain': 'クロスドメイン（グローバル市場）',
    'inputs': ['株価', '為替レート', 'コモディティ価格',
               '競合企業のアナウンス'],
    'function': '市場レジームシフトの検知、為替・コモディティの予測、'
                '競合のアクションへのアラート',
},
'Agent_06_Regulatory_Radar': {
    'type': '非身体化',
    'behavior': '予見型',
    'domain': 'クロスドメイン（グローバル規制）',
    'inputs': ['EU 規制フィード', '米国商務省のアクション',
               '経産省のアナウンス', 'WTO 紛争提訴'],
    'function': '規制変更を 30〜90 日前に予測、'
                'DPP コンプライアンスギャップの計算',
},
'Agent_07_Geopolitical_Oracle': {
    'type': '非身体化',
    'behavior': '予見型',
    'domain': '地政学リスク',
    'inputs': ['GPR 指数', 'EPU 指数', '制裁リスト',
               'SC 機密脅威インテリジェンス（暗号化）'],
    'function': '地政学リスクの進化を監視、制裁を予測、'
                'サプライチェーン経路の実行可能性を評価',
},
'Agent_08_Cyber_Guardian': {
    'type': '非身体化＋身体化',
    'behavior': '反応型＋予見型',
    'domain': 'サイバーセキュリティ',
    'inputs': ['SIEM アラート', 'ネットワークトラフィックパターン',
               'ダークウェブ監視', 'SC 機密脅威インテリジェンス'],

    'function': 'APT 活動の検知、攻撃ベクトルの予測、'
           '自動 SDN セグメンテーションのトリガー',
},
'Agent_09_Brand_Watcher': {
    'type': '非身体化',
    'behavior': '反応型',
    'domain': 'ブランド＆評判',
    'inputs': ['SNS センチメント（5 市場）', 'メディア報道',
               '偽造品検知アラート'],
    'function': 'ブランドの健全性をリアルタイムで追跡、偽造品急増の検知、'
                'PR 対応プロトコルのトリガー',
},
'Agent_10_Supply_Chain_Navigator': {
    'type': '非身体化＋身体化',
    'behavior': '予見型',
    'domain': 'クロスドメイン（サプライチェーン）',
    'inputs': ['サプライヤー健全性スコア', '出荷データ', '天気予報',
               '港湾混雑指数'],
    'function': 'サプライチェーンの経路最適化、混乱の予測、'
                '在庫バッファの管理',
},
'Agent_11_Financial_Controller': {
    'type': '非身体化',
    'behavior': '予見型',
```

```
    'domain': '財務企画',
    'inputs': ['日次 P&L', 'キャッシュフロー', '為替ポジション',
               'コモディティヘッジポジション'],
    'function': 'G-RAROC を日次で計算、資本配分の最適化、'
                'ヘッジ調整のトリガー',
},
'Agent_12_Talent_Compass': {
    'type': '非身体化',
    'behavior': '予見型',
    'domain': '人的資本',
    'inputs': ['従業員満足度調査', '離職率データ',
               '採用パイプライン', '研修完了率'],
    'function': 'キーパーソンの離職予測、人材配置の最適化、'
                '内部脅威指標のフラグ立て',
},
'Agent_13_Innovation_Scout': {
    'type': '非身体化',
    'behavior': '予見型',
    'domain': 'R&D・テクノロジー',

        'inputs': ['特許出願', '学術論文',
                   '競合の R&D アナウンス', '材料科学のブレークスルー'],
        'function': '技術的破壊の機会と脅威を特定、'
                    'R&D 投資の優先順位を推奨',
    },
    'Agent_14_Orchestrator': {
        'type': '非身体化',
        'behavior': '予見型',
        'domain': 'クロスドメイン（メタエージェント）',
        'inputs': ['他の 13 体すべてのエージェントの出力'],
        'function': 'エージェント・ドメイン間の創発現象（リー括弧）を検知、'
                    'エージェント間の矛盾する推奨の解決、'
                    'CEO 向けの統合戦略助言の生成',
    },
}

print(f"配備エージェント総数：{len(agent_roster)}")
print(f" 身体化：{sum(1 for a in agent_roster.values() if '身体化' in a['type'] and '非' not in
↪  a['type'])}")
print(f" 非身体化：{sum(1 for a in agent_roster.values() if a['type'] == '非身体化')}")
print(f" ハイブリッド：{sum(1 for a in agent_roster.values() if '＋' in a['type'])}")
```

解説

3 軸分類：

- 軸 1：身体化（物理センサーデータ）vs 非身体化（市場・規制データ）
- 軸 2：反応型（イベントに対応） vs 予見型（イベントを予測）
- 軸 3：単一ドメイン vs クロスドメイン

14 体の構成：

- Agent 01 —— 04：4 工場（ベトナム、トルコ、イタリア、日本）の身体化エージェント
- Agent 05 —— 13：市場、規制、地政学、サイバー、ブランド、サプライチェーン、財務、人材、技術の 9 ドメイン
- Agent 14：オーケストレータ —— 他の 13 体の出力を統合し、エージェント間のリー括弧を検出

Agent 14 の特殊役割：エージェント・ドメイン間の創発現象（リー括弧）を検知するメタエージェント。 個々のエージェントが見逃す「複数ドメインの相互作用による新しい脆弱性」を捉える。 これが GI 理論 Vol.2 の二層多様体構造（$\mathcal{M}_{\text{env}}$ は全エージェント共有、 $\mathcal{M}_{\text{weights}}^{(i)}$ はエージェントごと）の運用形態。

3 層モニタリングシステム：

- レベル 1（自動）：信頼区間超過時の自動フラグ
- レベル 2（監視）：橘によるダッシュボード監視
- レベル 3（経営）：CEO 週次ブリーフィング、取締役会月次ディープダイブ

13　形式検証と ZKML

13.1　Coq 証明と Z3 SMT 検証

背景

商業案件で前例のない最後のステップ —— 重要な安全性性質の形式検証（GI 理論第 24 —— 27 章）。 リームの主張：「我々のエージェントは 3,200 億円の売上と 12,000 人の雇用に影響する。 『Python のコードは動きますから信じてください』では許されない。数学的な証明が必要だ」。

コード

```
# ===== 第 25 章：Coq による形式検証 =====

# 検証する性質：「MC ドロップアウトの範囲内で多様体モデルが
# 正確であると仮定した場合、いかなるエージェントの推奨も、
# 企業価値を単月で 5%を超えて低下させることはない。」

# Coq の証明スケッチ（実際の Coq 構文での実装）：

coq_proof = r"""
Theorem agent_safety_bound :
  forall (z : manifold_point) (u : agent_recommendation),
    within_mc_dropout_bounds z ->
    enterprise_value (apply_recommendation z u) >=
    0.95 * enterprise_value z.
Proof.
  intros z u H_bounds.
  (* ステップ 1：推奨 u はポントリャーギンの最大値原理で計算されており、
     最適軌道に沿ってハミルトニアンの単調改善を保証する。*)
  apply pontryagin_monotonicity.
  (* ステップ 2：デコーダに対するリプシッツ上界（第 28 章、
     スペクトル正規化）が、潜在空間の小さな変化がデコード空間で
     有界な変化を生むことを保証する。*)
  apply spectral_norm_lipschitz_bound.
  (* ステップ 3：MC ドロップアウト上界が、真の状態が推定状態の
     イプシロン以内にあるという信頼性を提供する。*)
  apply mc_dropout_within_epsilon.
  exact H_bounds.
Qed.
"""

# Z3 SMT ソルバー検証（第 27 章）：
# 定義された動作範囲内のいかなる入力の組み合わせによっても
# エージェントの安全制約が違反されえないことを検証する。

from z3 import *

# 変数を定義

z = [Real(f'z_{i}') for i in range(8)]   # 潜在状態
u = [Real(f'u_{i}') for i in range(5)]   # 制御入力

# 安全制約を定義
enterprise_value = Function(
    'EV',
```

```
    RealSort(), RealSort(), RealSort(), RealSort(),
    RealSort(), RealSort(), RealSort(), RealSort(), RealSort()
)

solver = Solver()
# 制約追加：推奨は企業価値を 5%を超えて低下させてはならない
# solver.add(enterprise_value(*z_after) >= 0.95 * enterprise_value(*z_before))
# ...（完全な Z3 エンコーディング）

result = solver.check()
print(f"Z3 安全性検証：{result}")      # 期待値：sat（制約充足可能）
```

解説

検証する性質：「MC ドロップアウトの範囲内で多様体モデルが正確であると仮定した場合、いかなるエージェントの推奨も、企業価値を単月で 5% を超えて低下させることはない」。

Coq 証明の構造（コード中の `coq_proof` 文字列に格納）：

1. ポントリャーギン単調性：推奨は最大値原理で計算され、ハミルトニアンの単調改善を保証
2. スペクトル正規化リプシッツ上界（第 28 章）：潜在空間の小変化が有界変化を生む
3. MC ドロップアウト上界：真の状態が推定状態の ϵ 以内にある信頼性

これら 3 つの補題から `agent_safety_bound` 定理が導出される。

Z3 SMT の役割：Coq の定理証明は論理的健全性を保証するが、Z3 は動作範囲内の入力空間に対する制約充足性を検証する。両者は補完関係にある。

形式検証の総合結果：

- Coq 証明：検証済み √（安全性上界定理が証明された）
- Z3 SMT：充足可能 √（動作範囲内で制約違反は不可能）
- Agda 依存型：エージェントの挙動が型チェックに合格 √

検証された 4 つの性質：

1. 単一の推奨が月あたりの企業価値を 5% 超低下させることはない
2. エージェント推奨の組み合わせが多様体の滑らかさを保存する
3. 機密データの区画がエージェント境界を越えてリークすることはない
4. ゲージ変換のローテーションスケジュールが暗号論的に健全である

13.2 ZKML 証明生成

背景

最後の信頼メカニズム（GI 理論第 4 章）。綾鳥の取締役会は多様体の計算が正しいことの保証を必要としていたが、リームは訓練済み VAE モデルもそれに組み込まれた機密データも開示できない。ゼロ知識機械学習（ZKML）がその解決策。

コード

```
# ===== 第 4 章：ZKML 証明の生成 =====
# 綾鳥に対し、多様体の計算が正しいことを証明する
# —— 訓練済みモデルの重みや機密データを明かすことなく
```

```
# ステップ 1：デコーダを ONNX にエクスポート
# ステップ 2：算術回路に変換（ezkl）
# ステップ 3：証明を生成
# ステップ 4：クライアントが証明を検証

# クライアント側の検証は 1 秒未満で完了し、モデル、データ、
# SC 機密情報へのアクセスは一切不要。

print("ZKML 証明生成：")
print(" 回路サイズ：240 万ゲート")
print(" 証明生成時間：47 分")
print(" 証明サイズ：1.2 MB")
print(" 検証時間：0.8 秒")
print(" 結果：証明有効 ✓")
print()
print("綾鳥の取締役会は以下を検証できるようになった：")
print(" 1. 多様体が記載された入力から正しく計算されたこと")
print(" 2. データの捏造や誤情報の混入がないこと")
print(" 3. 数学的操作（曲率、測地線）が正しいこと")
print(" —— モデルの重みや機密インテリジェンスを見ることなく")
```

解説

ZKML（Goldwasser-Micali-Rackoff の対話的証明系の機械学習適用）の 3 性質：

- **完全性**：計算が正しければ検証者は必ず納得できる
- **健全性**：計算をごまかしていれば検証者は必ず検出できる
- **零知識性**：検証者は計算の正しさ以外の情報を得られない

ezkl パイプライン：

1. デコーダを ONNX にエクスポート
2. 算術回路に変換
3. 証明を生成
4. クライアントが証明を検証

ケーススタディでの実測：

- 回路サイズ：240 万ゲート
- 証明生成時間：47 分（メリディアン側、1 回限り）
- 証明サイズ：1.2 MB
- 検証時間：0.8 秒（クライアント側、毎回）

綾鳥取締役会が検証可能になった事項：

1. 多様体が記載された入力から正しく計算されたこと
2. データの捏造や誤情報の混入がないこと
3. 数学的操作（曲率、測地線）が正しいこと

—— モデルの重みや機密インテリジェンスを見ることなく。

ZKML の限界：ZKML は「計算過程に嘘がない」ことを保証するが、「パラメータが正当な訓練プロセスで得られたものか」は保証しない。このため GI 理論は 3 つの防御層を要求する：(1) ZKML（計算過程）、(2) 訓練ログのコミットメント（パラメータの出自）、(3) 第三者監査（プロセス全体の公正性）。

A　推奨ライブラリ一覧

本書のコード片で使用される Python ライブラリと、推奨バージョンを示す。 バージョンは 2026 年 4 月時点での安定版を基準とした。

A.1　基本数値計算・データ処理

- `numpy`（≥ 1.26）—— 線形代数、配列操作の中核
- `scipy`（≥ 1.13）—— 統計、最適化、線形代数の応用関数群（`scipy.stats.special_ortho_group` は `gauge_transform` で使用）
- `pandas`（≥ 2.2）—— データフレーム処理
- `scikit-learn`（≥ 1.4）—— KNN 補完、ロバストスケーリング、最近傍探索

A.2　深層学習と Neural ODE

- `torch`（≥ 2.2）—— PyTorch、VAE 実装と自動微分の基盤
- `torchdiffeq`（$\geq 0.2.4$）—— Neural ODE の数値積分器（`constrained_dynamics`、`manifold_optimal_control` で使用）

A.3　説明可能性と因果推論

- `shap`（≥ 0.45）—— Shapley 値ベースの寄与度分析
- `econml`（≥ 0.15）—— DML（Double Machine Learning）実装

A.4　位相的データ解析

- `ripser`（≥ 0.6）—— Vietoris-Rips フィルトレーション、パーシステント・ホモロジー
- `persim`（≥ 0.3）—— パーシステンス図の可視化と距離計算

A.5　形式検証

- `z3-solver`（≥ 4.13）—— Microsoft Research の SMT ソルバー
- Coq Proof Assistant（≥ 8.18）—— 定理証明（Python 外、別途インストール）
- Agda（≥ 2.7）—— 依存型理論プログラミング（同上）

A.6　ZKML

- `ezkl`（≥ 11.0）—— ニューラルネットワーク向けゼロ知識証明
- `onnx`（≥ 1.15）—— ニューラルネットワークのモデル交換フォーマット

A.7　可視化（本書のコードでは直接使用しないが、本編で言及）

- `matplotlib`, `plotly` —— 2D／3D プロット
- `three.js` 等（フロントエンド） —— Project Mirror ダッシュボード

A.8　インストールコマンド

```
# 基本セット
pip install numpy scipy pandas scikit-learn

# 深層学習
pip install torch torchdiffeq
```

```
# 説明可能性・因果推論
pip install shap econml

# TDA
pip install ripser persim

# 形式検証
pip install z3-solver

# ZKML
pip install ezkl onnx
```

B　コード索引

各コード片を関数名・対応する本編シーン・対応する数学概念で索引化する。

コードファイル	本編シーン	数学概念
preprocess_data_loading	第 1 月、データ統合	100 指標 × 8 カテゴリの構造化
preprocess_function	第 1 月、リンの懸念	KNN 補完、レジームシフト保存
estimate_intrinsic_dimension	第 1 月、潜在次元決定	Levina-Bickel 最尤推定
proposition_check_first	第 1 月、最初の数学的危機	命題 1.1 の 3 条件（不合格）
constrained_dynamics	第 1 月、5 日間データ手術	ホイットニー拡張定理、制約付き Neural ODE
proposition_check_second	第 1 月、危機解決	命題 1.1 の 3 条件（合格）
compute_pullback_metric	第 2 月、地形図出現	引き戻し計量 $g_{ij} = (J^\top J)_{ij}$
compute_scalar_curvature	第 2 月、不安定領域同定	スカラー曲率、リッチ縮約
compute_lie_derivative	第 2 月、ベトナム工場停止	リー微分 $\mathcal{L}_X g$
compute_lie_bracket	第 2 月、創発現象検出	リー括弧 $[X, Y]$、致命的一撃
mc_dropout	全期間	ベイズ近似、不確実性定量化
shap_dml	第 5 月、偽造品危機	Shapley 値、Double ML
manifold_optimal_control	第 2 月、緊急対応計画	ポントリャーギン最大値原理、クリストッフェル補正
control_options	第 2 月、98 億円プラン	7 つの制御オプション、最適経路求解
sheaf_detection	第 5 月／第 8 月	セル層、層ラプラシアン、不整合検知
x_dpp_regulation	第 5 月、規制の津波	DPP ベクトル場
surgery_theory	第 5 月、サプライチェーン再構築	手術理論、コボルディズム条件
x_takeover	第 6 —— 7 月、敵対的買収	4 方向リー括弧
tda_persistent_homology	第 6 —— 7 月、シェル会社検出	パーシステント・ホモロジー H_0, H_1
gauge_transform	第 8 月、内部漏洩後	ゲージ変換、座標不変性、特殊直交群
stress_tensor	第 10 月、二重ショック	応力テンソル、応力集中比
stormer_verlet	第 10 月、5 年予測	シンプレクティック積

B.1　パイプラインステップ別の主要コード片

- **ステップ 1（データ前処理）** —— `preprocess_data_loading`, `preprocess_function`
- **ステップ 2（VAE 多様体構築）** —— `estimate_intrinsic_dimension`, `proposition_check_first`, `constrained_dynamics`, `proposition_check_second`
- **ステップ 3（プルバック計量）** —— `compute_pullback_metric`
- **ステップ 4（曲率）** —— `compute_scalar_curvature`
- **ステップ 5（リー微分・リー括弧）** —— `compute_lie_derivative`, `compute_lie_bracket`
- **ステップ 6（MC ドロップアウト）** —— `mc_dropout`
- **ステップ 7（SHAP / DML）** —— `shap_dml`
- **ステップ 8（最適制御）** —— `manifold_optimal_control`, `control_options`
- **幾何学的拡張** —— `sheaf_detection`, `x_dpp_regulation`, `surgery_theory`, `x_takeover`, `tda_persistent_homology`, `gauge_transform`, `stress_tensor`, `stormer_verlet`
- **エージェント配備・形式検証** —— `agent_roster`, `formal_verification_coq_z3`, `zkml_proof`

B.2　結語

本書に掲載されたすべてのコード片は、ジオメトリック・インテリジェンス理論の数学的基盤と ケーススタディ本編の経営局面を結ぶ橋渡しを意図したものである。 これらは完全な実装ではなく、各組織の実環境に合わせた本格的なエンジニアリングが別途必要となる。

しかし、本書のコードが示しているのは 何が計算可能か の輪郭である。 リーマン幾何学・代数的位相幾何学・形式手法・暗号理論を企業経営の意思決定支援に統合することは、 今日の Python エコシステムで実現可能である —— これが本書の伝えるべき主要な事実である。

— Étale Cohomology, 2026 年 4 月